

CryptoLens

Automated Cryptocurrency Market Data Pipeline

Phase 1 Project Proposal (Updated)

Field	Detail
Course	Data Analysis and Visualizations — 5th Semester
Team Members	Abdullah Saeed (24L-2529); Arqum Umais (24L-2591)
GitHub Repository	https://github.com/x-abdullah-saeed-x/CryptoLens
Document Version	Phase 1 — Revised after data-sourcing & ingestion review

1. Problem Statement

Cryptocurrency markets generate high-frequency, continuously evolving trading data across hundreds of pairs. Traders, analysts, and researchers need a reliable, query-ready historical and incrementally-updated dataset to study price trends, volatility, and market-structure changes such as new listings, delistings, and trading halts. Raw exchange data arrives as large archived flat files and live REST responses that are not directly analysis-ready. CryptoLens builds an automated pipeline that ingests, cleans, conforms, and aggregates this data into a warehouse suitable for BI dashboards and ad hoc analysis.

2. Objectives

- Build a reproducible ingestion pipeline sourcing real historical and incremental cryptocurrency market data — not synthetic/generated samples.
- Implement a Medallion (Bronze → Silver → Gold) architecture on Databricks to progressively clean, conform, and aggregate raw data.
- Track a broad, dynamically-selected universe (top 50 USDT pairs by volume) at 1-minute granularity for meaningful analytical depth and realistic data volumes.
- Correctly model slowly changing reference data (trading-pair status) using real MERGE-based inserts, updates, and soft-deletes — not an append-only load.
- Deliver a Power BI dashboard surfacing price trends, volume, and volatility across tracked symbols.

3. Data Sources

Source	Purpose
data.binance.vision	Binance's public historical market-data archive. Provides monthly and daily ZIP-compressed kline (candlestick) files per symbol/interval, each with a SHA256 .CHECKSUM sidecar file for integrity verification.
api.binance.com — /api/v3/ticker/24hr	Live endpoint used to rank all USDT pairs by 24-hour quote volume and dynamically select the top 50 to track (no hardcoded symbol list).
api.binance.com — /api/v3/exchangeInfo	Live endpoint used to source dim_symbol reference data — trading status, listing/delisting, and trading filters. This is the one data source that genuinely changes over time, driving real MERGE upserts and soft-deletes.

4. Load Strategy

4.1 Full Load

One monthly 1-minute kline file per tracked symbol (~43,200 rows/symbol/month), downloaded for all top-50 USDT pairs. Expected aggregate size: ~200+ MB.

4.2 Incremental Load

One daily 1-minute kline file per tracked symbol (~1,440 rows/symbol/day), refreshed on every pipeline run. Expected aggregate size: ~10 MB/day.

4.3 Integrity Verification

Every downloaded ZIP is hashed (SHA256) and compared against Binance's official .CHECKSUM file before being trusted and extracted, guarding against corrupted or tampered downloads.

5. Bronze Ingest Process

The bronze stage (01_bronze_ingest) performs the following steps:

- Resolve the symbol universe dynamically via /ticker/24hr (top 50 USDT pairs by volume) instead of a hardcoded list.
- For each symbol, build the monthly URL (full load) and daily URL (incremental load) per the data.binance.vision path convention, download the ZIP, verify its SHA256 against the .CHECKSUM sidecar, and extract the raw CSV.
- Land verified raw CSVs into the Bronze layer (a Databricks Unity Catalog Volume), tagged with symbol, interval, and ingestion timestamp.
- Separately snapshot /exchangeInfo and land it as its own bronze source, feeding the dim_symbol MERGE logic in Silver.

Environment note: Databricks Free Edition's serverless compute restricts outbound internet access to an internal allowlist that does not include Binance's domains. The download + checksum-verification step is therefore executed on a local machine, and the verified files are pushed into the Databricks Volume via the Databricks CLI (databricks fs cp). This is documented as an environment-specific step in the repo README rather than a workaround hidden from the pipeline design.

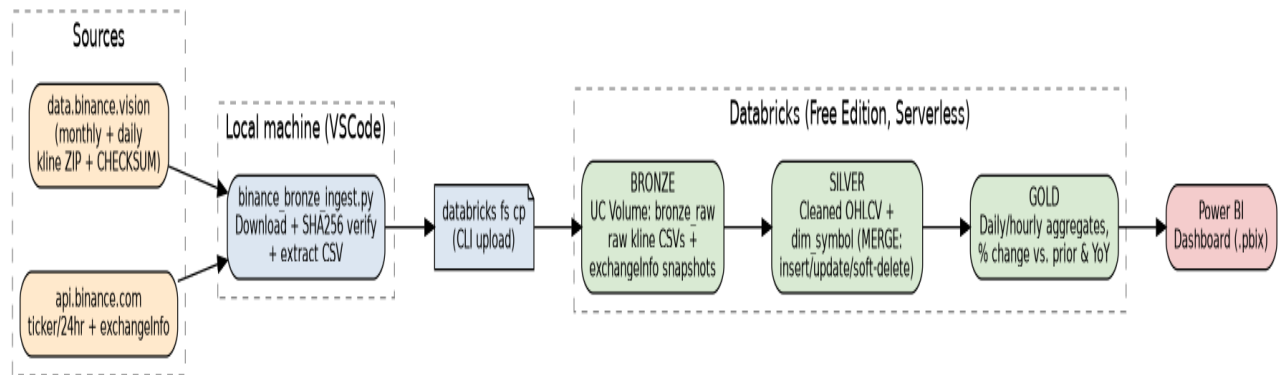
6. Silver Layer

- Parse and clean kline CSVs: assign proper column names (open_time, open, high, low, close, volume, close_time, quote_asset_volume, number_of_trades, taker_buy_base_volume, taker_buy_quote_volume), convert epoch timestamps, cast numeric types, and deduplicate.
- Build a conformed dim_symbol table via MERGE from exchangeInfo snapshots — handling inserts for new listings, updates for status/filter changes, and soft-deletes (is_active flag) for delisted symbols.

7. Gold Layer

- Aggregate OHLCV metrics by symbol/day/hour.
- Compute percent change versus the prior period and versus the year-ago period, and rolling volatility.
- Expose segment- and symbol-level summary tables to Power BI.

8. Architecture Overview



End-to-end flow: Binance sources → local download/checksum (Free Edition workaround) → Bronze Volume → Silver (clean + dim_symbol MERGE) → Gold aggregates → Power BI.

9. Tools & Technologies

Category	Tool
Data warehouse / compute	Databricks (Free Edition, Serverless SQL & Compute), Delta Lake, Unity Catalog Volumes
Processing	PySpark, Spark SQL
Ingestion scripting	Python (requests, hashlib, zipfile) — run locally, uploaded via Databricks CLI
Dashboarding	Power BI
Version control	GitHub — https://github.com/x-abdullah-saeed-x/CryptoLens

10. Revised Timeline

Week	Task
1	Finalize data sources; build and test binance_bronze_ingest.py locally; verify checksums.
2	Set up Databricks Unity Catalog Volume; upload bronze data via CLI; build 01_bronze_ingest landing logic.
3	Build 02_silver_transform: OHLCV cleaning + dim_symbol MERGE from exchangeInfo.
4	Build 03_gold_aggregate: rollups, % change vs. prior/year-ago periods.
5	Build Power BI dashboard; connect to Gold tables.
6	Testing, documentation, README updates, final submission.

11. Team & Responsibilities

Member	Responsibility
Abdullah Saeed (24L-2529)	Data ingestion & Bronze layer: source selection, download/checksum scripting, Databricks Volume setup, CLI upload workflow.
Arqum Umais (24L-2591)	Silver/Gold transformations and dashboarding: data cleaning, dim_symbol MERGE logic, aggregate design, Power BI development.

Both members jointly handle testing, documentation, and repository maintenance.

12. Risks & Mitigations

Risk	Mitigation
Databricks Free Edition serverless compute blocks outbound calls to Binance domains.	Download and checksum-verify locally; push verified files into the Unity Catalog Volume via the Databricks CLI.
Binance API rate limiting during bulk downloads across 50 symbols.	Sequential requests with per-symbol error handling; failures are logged and skipped rather than aborting the whole run.
Newly-listed symbols lack a full month of historical data.	Downloader catches and logs these as skipped (observed: HYPEUSDT, MARSCOINUSDT) without failing the overall load.
Repository bloat from committing large raw data files.	Bulk full_load/incremental_load data is .gitignored; only one representative symbol's checksum-verified sample pair is committed.